

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
про виконання лабораторної роботи № 2
« Структури даних та функціональна декомпозиція »

Виконала:
студентка 3 курсу
групи ФЕП-31
Антошкова Аліна Олександрівна
Перевірів:
Доц. Рабош Роман Васильович

Мета: Сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпонувати програму на невеликі функції з чітко визначеною відповідальністю.

Варіант 12

Постановка завдання

Вхідні дані містять операції з полями: дата, категорія, сума та тип (income/expense). Реалізувати модуль обробки структурованих даних, що забезпечує: підрахунок загальної суми доходів, загальної суми витрат, розрахунок балансу, обчислення витрат за категоріями, пошук найбільшої витрати, підрахунок частоти категорій за допомогою Counter, фільтрацію операцій за сумою через closure, групування операцій за категоріями через defaultdict. Оцінити часову складність щонайменше трьох операцій, реалізувати параметризовані виклики (*args, **kwargs), декоратор заміру часу та провести бенчмарк швидкості лінійного пошуку в list проти пошуку в dict.

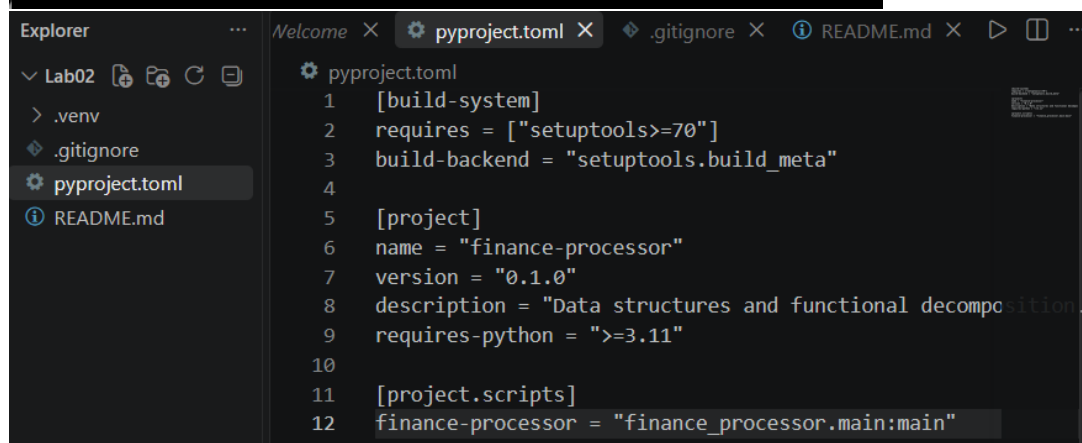
Хід роботи:

1. Додавання файлів базової конфігурації Lab02 (.gitignore, pyproject.toml, README.md) та виконання першого коміту

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab02 (main)
$ git add .gitignore pyproject.toml README.md

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab02 (main)
$ git commit -m "Lab02: initial commit, setup project configuration"
[main 6cc5c15] Lab02: initial commit, setup project configuration
3 files changed, 31 insertions(+)
create mode 100644 Lab02/.gitignore
create mode 100644 Lab02/README.md
create mode 100644 Lab02/pyproject.toml

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab02 (main)
$ git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 921 bytes | 921.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/AntoskovaA/paython.git
76258dc..6cc5c15  main -> main
```



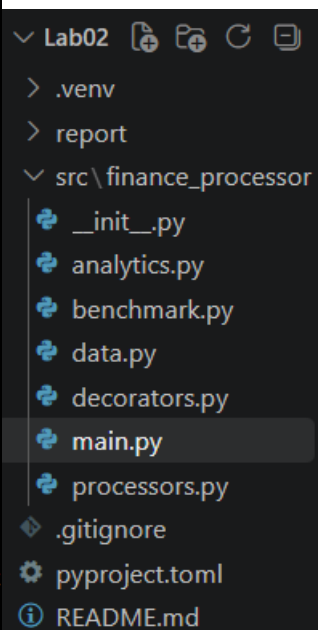
2. Створення віртуального оточення .venv та формування файлової структури пакетів за принципом src-layout.

```
PS C:\Users\Alina\Desktop\пайтон\paython\Lab02> python -m venv .venv
PS C:\Users\Alina\Desktop\пайтон\paython\Lab02> .venv/Scripts/activate
```

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab02 (main)
$ git add src/

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab02 (main)
$ git commit -m "create src-layout package structure and modules"
[main dfffb15] create src-layout package structure and modules
7 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Lab02/src/finance_processor/__init__.py
create mode 100644 Lab02/src/finance_processor/analytics.py
create mode 100644 Lab02/src/finance_processor/benchmark.py
create mode 100644 Lab02/src/finance_processor/data.py
create mode 100644 Lab02/src/finance_processor/decorators.py
create mode 100644 Lab02/src/finance_processor/main.py
create mode 100644 Lab02/src/finance_processor/processors.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab02 (main)
$ git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 530 bytes | 530.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object
To https://github.com/AntoskovaA/paython.git
6cc5c15..dfffb15 main -> main
```



3. Реалізація функціональної декомпозиції, замикань, декоратора та операцій над структурами даних у модулях processors.py та analytics.py.

```
decorators.py X processors.py X analytics.py X benchmark.py X main.py X .gitig D
src > finance_processor > decorators.py > measure_time
1 from functools import wraps
2 from time import perf_counter
3 from typing import Any, Callable
4
5
6 def measure_time(func: Callable) -> Callable:
7     """Декоратор для заміру часу виконання функції зі збереженням метаданих через
8     @wraps(func)
9     def wrapper(*args: Any, **kwargs: Any) -> Any:
10         start = perf_counter()
11         result = func(*args, **kwargs)
12         elapsed = perf_counter() - start
13         print(f"[Таймінг] {func.__name__}: {elapsed:.8f} s")
14         return result
15
16     return wrapper
```

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab02 (main)
$ git add .

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab02 (main)
$ git commit -m "Lab02: implement data structures, collections, closures and decorators"
[main 5ca9d97] Lab02: implement data structures, collections, closures and decorators
4 files changed, 105 insertions(+)

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab02 (main)
$ git push origin main
Enumerating objects: 17, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 12 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (9/9), 2.78 KiB | 949.00 KiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/AntoskovaA/paython.git
dfffb15..5ca9d97 main -> main
```

4. Реалізація бенчмарку (порівняння list search та dict search) та головної точки входу main.py.

```
benchmark.py X main.py X .gitignore X README.md X
src > finance_processor > benchmark.py > run_benchmark
1 from time import perf_counter
2
3
4 def run_benchmark() -> None:
5     sizes = [1_000, 10_000, 100_000]
6     print("\n--- Результати експериментального Benchmark ---")
7     print(f"{'Кількість records':<18} | {'list search (c)':<18} | {'dict search (c)':<18}")
8     print("-" * 60)
9
10    for size in sizes:
11        dataset = [
12            {"id": i, "category": "Тест", "amount": 100.0, "type": "expense"}
13            for i in range(size)
14        ]
15        index = {item["id"]: item for item in dataset}
16        target_id = size - 1 # Пошук останнього елемента (найгірший сценарій)
17
18        # Лінійний пошук у списку O(n)
19        t_start = perf_counter()
20        _ = next((x for x in dataset if x["id"] == target_id), None)
21        t_list = perf_counter() - t_start
22
23        # Хеш-пошук у словнику O(1)
24        t_start = perf_counter()
25        _ = index.get(target_id)
26        t_dict = perf_counter() - t_start
27
28        print(f"{'size':<18} | {'t_list:<18.8f'} | {'t_dict:<18.8f}')
```

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/python/Lab02 (main)
$ git add src/finance_processor/benchmark.py src/finance_processor/main.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/python/Lab02 (main)
$ git commit -m "Lab02: add benchmark measurements and entryptpoint"
[main df46e40] Lab02: add benchmark measurements and entryptpoint
2 files changed, 101 insertions(+)

Alina@ASUS MINGW64 ~/Desktop/пайтон/python/Lab02 (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 2.17 KiB | 2.17 MiB/s, done.
Total 7 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/AntoskovaA/python.git
5ca9d97..df46e40 main -> main
```

5. Встановлення пакета в режимі редагування (editable installation) через pip.

```
(.venv) PS C:\Users\Alina\Desktop\пайтон\paython\Lab02> python -m pip install -e .
Obtaining file:///C:/Users/Alina/Desktop/%D0%BF%D0%B0%D0%B9%D1%82%D0%BE%D0%BD/paython/Lab02
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: finance-processor
Building editable for finance-processor (pyproject.toml) ... done
Created wheel for finance-processor: filename=finance_processor-0.1.0-0.editable-py3-none-any.whl size=1666 sha256=17d94bb29f786ed12fe6f1a90537ef8ff2892909fe7ca17b01f44e9ba1861dc7
Stored in directory: C:\Users\Alina\AppData\Local\Temp\pip-ephem-wheel-cache-82cwbsj\wheels\56\3a\57\d07a52cc7fd04994ba77996b58bf3d7b2e68122efd46b935bb
Successfully built finance-processor
Installing collected packages: finance-processor
Successfully installed finance-processor-0.1.0
```

6. Результати виконання програми: вивід структур даних, агрегацій, роботи декоратора, closure та запуск бенчмарку.

```
(.venv) PS C:\Users\Alina\Desktop\пайтон\paython\Lab02> python -m finance_processor.main
Розрахований баланс: 27380.00 грн

Витрати за категоріями: {'Продукти': 2370.0, 'Транспорт': 350.0, 'Комунальні': 2100.0, 'Розваги': 800.0}

[Lambda] Найбільша витрата: Комунальні на суму 2100.0 грн

[Counter] Кількість операцій у кожній категорії: {'Зарплата': 1, 'Продукти': 2, 'Транспорт': 1, 'Фриланс': 1, 'Комунальні': 1, 'Розваги': 1}

[defaultdict] Групування операцій за категоріями:
* Зарплата: 1 запис(ів)
* Продукти: 2 запис(ів)
* Транспорт: 1 запис(ів)
* Фриланс: 1 запис(ів)
* Комунальні: 1 запис(ів)
* Розваги: 1 запис(ів)

[Closure] Операції на суму >= 1500 грн:
- 2026-09-01 | Зарплата | 28000.0 грн (income)
- 2026-09-10 | Фриланс | 5000.0 грн (income)
- 2026-09-15 | Комунальні | 2100.0 грн (expense)

[*args] Коригування суми: 86.50 грн
[**kwargs] Створено запис: {'id': 8, 'date': '2026-09-20', 'category': 'Книги', 'amount': 450.0, 'type': 'expense'}

[Dict comprehension] Швидкий доступ до ID=3: {'id': 3, 'date': '2026-09-05', 'category': 'Транспорт', 'amount': 350.0, 'type': 'expense'}
```

```
--- Результати експериментального Benchmark ---
Кількість records | list search (c) | dict search (c)
-----
1000                | 0.00005560      | 0.00000150
10000               | 0.00060690      | 0.00000240
100000              | 0.00793230      | 0.00001700
```

7. Верифікація стану Git-репозиторію за допомогою команд `git log --oneline` та `git status`.

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab02 (main)
$ git log --oneline
df46e40 (HEAD -> main, origin/main) Lab02: add benchmark measurements and entrypoint
5ca9d97 Lab02: implement data structures, collections, closures and decorators
dffffb15 create src-layout package structure and modules
6cc5c15 Lab02: initial commit, setup project configuration
76258dc saved report as pdf
fc4c505 modified report
2310b6a added report
41d948a Add main entry point and README
74be2b5 Add transaction model and calculation services
ecb0c51 created folders for project, init files and virtual environment
2215b9f initial commit, setup project

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab02 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Опис структури даних та обґрунтування вибору колекцій

- **list:** використано для зберігання основної колекції транзакцій `transactions` як послідовного змінюваного журналу операцій, що гарантує збереження хронологічного порядку.
- **tuple:** застосовано у вигляді `ALLOWED_TYPES = ("income", "expense")` для зберігання константних значень допустимих типів операцій, які не мають змінюватися в процесі виконання.
- **set:** використано у функції `get_unique_categories` для швидкого знаходження множини унікальних найменувань категорій та виключення дублікатів за час $O(n)$.
- **dict:** обрано як формат представлення окремої транзакції (зручність доступу за ключами-атрибутами), а також для побудови швидкого хеш-індексу за `id` із середньою складністю пошуку $O(1)$.
- **Counter:** застосовано у `count_category_frequencies` для прямого підрахунку розподілу кількості транзакцій за категоріями без зайвого коду.
- **defaultdict:** застосовано в `group_transactions_by_category` для чистого групування операцій за списками без ручної ініціалізації ключів.

Опис створених модулів

- **data.py** — модуль визначення початкових структурованих наборів даних та незмінних константних кортежів.

- **decorators.py** — модуль технічних модифікаторів функцій; містить декоратор `measure_time` для заміру часу виконання із збереженням сигнатури через `functools.wraps`.
- **processors.py** — модуль роботи з колекціями (`set`, `Counter`, `defaultdict`), що реалізує сортування, групування та генераторні вирази над транзакціями.
- **analytics.py** — модуль функціональної декомпозиції, агрегації фінансових показників, замикань (`closure`) для фільтрації та обробки довільних аргументів `*args`, `**kwargs`.
- **benchmark.py** — експериментальний модуль замірів швидкості пошуку у списку та словнику на великих масивах даних (10^3 , 10^4 , 10^5 елементів).
- **main.py** — головна точка входу проєкту, яка послідовно викликає розроблені компоненти та демонструє отримані результати в консолі.

Аналіз часової складності операцій (Big-O)

Операція у коді	Реалізація	Складність	Обґрунтування
Пошук операції в <code>list</code> DOCX	<pre>next(x for x in dataset if x["id"] == target)</pre> DOCX	$O(n)$ DOCX	Потребує повного послідовного перебору списку від першого до шуканого елемента. DOCX
Побудова індексу <code>dict</code> DOCX	<pre>{item["id"]: item for item in dataset}</pre> DOCX	$O(n)$ DOCX	Одноразовий прохід по всіх n елементах із вставкою в хеш-таблицю. DOCX
Пошук операції в <code>dict</code> DOCX	<pre>index.get(target_id)</pre> DOCX	$O(1)$ DOCX	Прямий доступ до елемента за хеш-кодом ключа <code>id</code> без перебору масиву. DOCX
Виділення категорій у <code>set</code>	<pre>{item["category"] for item in items}</pre>	$O(n)$	Лінійний прохід по елементах із додаванням у хеш-множину за $O(1)$ у середньому. DOCX
Пошук найбільшої витрати DOCX	<pre>max(items, key=lambda x: x["amount"])</pre> DOCX	$O(n)$	Одноразовий повний перебір усіх записів для знаходження екстремуму. DOCX

Висновок:

Під час виконання лабораторної роботи № 2 було закріплено практичні навички професійного вибору та використання вбудованих і розширених структур даних у Python (`list`, `tuple`, `set`, `dict`, `Counter`, `defaultdict`), а також реалізовано функціональну декомпозицію програми.

На прикладі предметної області фінансового трекера (Варіант 12) побудовано систему аналізу транзакцій із чітким розділенням відповідальності між модулями. Успішно застосовано механізм замикань (closures) для гнучкої параметризації предикатів фільтрації за сумою, декоратор із бібліотекою `functools.wraps` для точного заміру часу виконання функцій, а також функції зі змінною кількістю аргументів `*args` та `**kwargs`.

В експериментальній частині проведено бенчмарк для оцінки теоретичної складності Big-O. Практичні заміри підтвердили, що лінійний пошук у `list` має складність $O(n)$ і його тривалість прямо пропорційно зростає зі збільшенням розміру вибірки, тоді як доступ через хеш-таблицю `dict` демонструє амортизовану швидкість $O(1)$, зберігаючи майже константний час доступу незалежно від обсягу даних (10^3 чи 10^5 записів). Проєкт організовано відповідно до стандарту `src-layout`, пакет інстальовано в режимі розробки, а всі етапи зафіксовано в системі контролю версій Git.